NK ETHICAL (IND) LTD - Web Platform

Workflow, Data Flow and System Design Decisions

Technical overview | 17 October 2026 | Prepared for NK ETHICAL (IND) LTD

This document explains, in simple words, how the platform works from end to end: what the visitor does, what each part of the system does with the data, and why each design choice was made. It is written for a reader who knows the business, not for a reader who will maintain the code.

1. What this system does

This is a company website with a contact form, plus the machinery that runs behind the form. A visitor reads the product list, sends an inquiry, and gets a reference number. Behind the scenes the inquiry is saved to a database first and then copied into a spreadsheet by itself. The spreadsheet can always be rebuilt from the database.

Three jobs, handled by three separate parts:

- Show the website. Fast, mostly plain pages with very little browser code.
- Receive the inquiry and store it safely.
- Copy stored inquiries into a spreadsheet, by itself, and never lose one.

2. The parts of the system

Part	Built with	Its one job
Website	Next.js 16, React 19, Tailwind 4	Show pages and the form. Pages are sent as ready-made HTML; the browser runs only a little code.
Website server	Next.js route handler (Node.js)	Own the public form address. Check the request again, then pass it to the intake service.
Intake service	Spring Boot 4.1, Java 25	Clean the data, check the data, save the data. This is the trusted gate.
Data store	PostgreSQL 16	Hold every inquiry. It is the one and only source of truth.
Export worker	Background job inside the intake service	Copy new inquiries into Excel and/or Google Sheets.
Documents	docs folder in the project	Architecture, security, export runbook and the API contract.

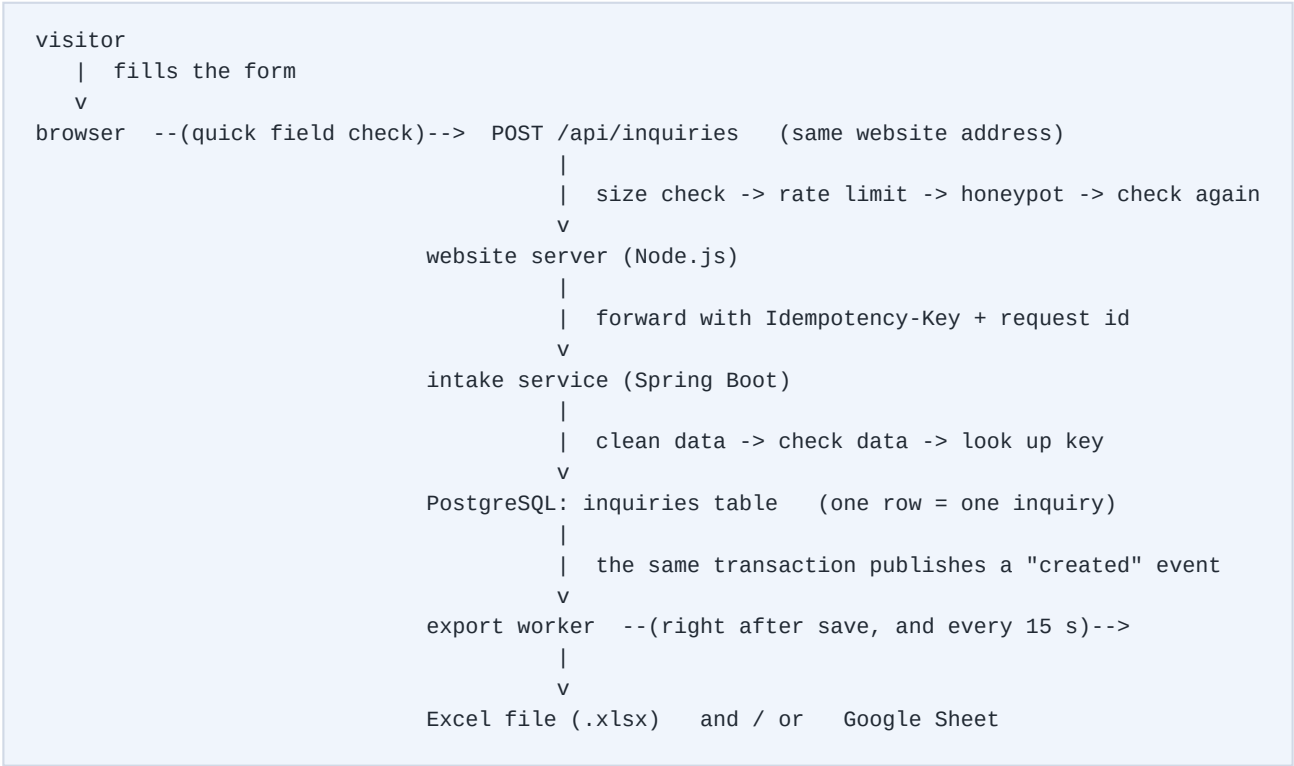
3. Step-by-step workflow

This is the life of one inquiry, from the visitor to the spreadsheet.

1. The visitor opens the site. The server sends finished HTML. The browser runs only the small amount of code the page needs.
2. The visitor fills the form and presses Send. The browser checks the fields first, for quick feedback. Nothing is sent while a field is clearly wrong.

3. The browser sends the form data to /api/inquiries on the same website address. It also sends a random Idempotency-Key and fills a hidden field that only a robot would touch (the honeypot).
4. The website server checks the request: it refuses anything larger than 16 KB before reading it; it counts requests per visitor address (10 per minute, 5 at once, 600 per minute for everyone) and answers 429 when there are too many; if the hidden field was filled it pretends to succeed and drops the request; it checks the data again with the same rules as the browser; then it forwards the data to the intake service with the same key and a request id.
5. The intake service adds a request id to the logs; applies its own size and rate checks; cleans the data first (removes markup and control characters, normalises text, cuts it to the allowed length); checks the cleaned data; then looks up the Idempotency-Key. If the key already exists it returns the same reference and saves nothing new.
6. If the key is new, the service saves exactly one row to PostgreSQL and answers 201 with the reference number. The save and the export trigger happen in one transaction, so a failed save can never start an export.
7. A background worker wakes up right after the save (and again every 15 seconds as a safety net). It takes up to 250 not-yet-exported rows and writes them to the spreadsheet.
8. The website shows the success panel with the reference number, and the visitor is done.

4. Data flow in one picture



5. What is stored (the inquiries table)

One row equals one inquiry. The row is written before the visitor is answered, so the reference number always points at data that really exists.

Group of columns	Example columns	Why it exists
Identity	id, public_id	id is private; public_id is the reference the visitor sees.
Repeat protection	client_request_id (unique)	The same request sent twice results in one row.
The inquiry	name, email, phone, company, message, consent	Cleaned data. Column sizes match the form limits exactly.
Export state	export_status, export_attempts, export_error, exported_at	Follows the copy to the spreadsheet.
Abuse facts	source_ip_hash, user_agent	A keyed code for the address, never the real address.
Bookkeeping	created_at, updated_at, version	Times, plus a counter that stops two workers overwriting each other.

6. How the spreadsheet copy works

Each row moves through a small set of states. A row can never be silently dropped; it is either exported or parked for a person.

```
PENDING  ->  PROCESSING  ->  EXPORTED
                  |
                  +-->  back to PENDING   (temporary problem: retry)
                  |
                  +-->  FAILED            (too many tries: a person must look)
```

1. The worker asks the database for the next batch: up to 250 rows that are PENDING and have been tried fewer than 6 times. It locks those rows so no other worker can take the same ones. Oldest row first, so the sheet stays in time order.
2. The rows become PROCESSING and the database connection is closed immediately. Slow spreadsheet work never holds a database connection.
3. The worker writes the batch to every configured sink: the Excel file, the Google Sheet, or both.
4. On success the rows become EXPORTED. Writing the same reference twice is skipped, so a retry never duplicates a row.
5. On failure the rows go back to PENDING, the try count goes up, and the reason is saved. After 6 failed tries a row becomes FAILED and an alarm should fire.

Safety nets

- A save triggers the worker immediately, so the spreadsheet is usually up to date within a moment.
- A timer runs every 15 seconds. It puts rows that were stuck in PROCESSING for more than 5 minutes

back into the queue (this covers a crash), then drains whatever is pending.

- The Excel file is written to a temporary name and then swapped in with one atomic move, so a reader never sees half a file.
- Two locks protect the Excel file: one inside the process and one across processes (a file lock). Both are needed, because they stop different problems.

7. When something goes wrong

Problem	What the system does
Database is down	Answers 503. Nothing is saved and nothing is exported. No half-done work.
The same request arrives twice	Returns the first reference. No second row is created.
Spreadsheet sink is down or the disk is full	Rows go back to PENDING and are retried later.
Six tries all fail	The row is parked as FAILED and counted, so a person is alerted.
The process is killed mid-export	After 5 minutes the timer puts the rows back in the queue.
The Excel file is open in Excel by a person	Treated as a temporary problem and retried.
The Excel file is damaged	Set aside and rebuilt from the database.

8. Why it was built this way (design decisions)

Decision	Why
Save to the database before answering the visitor	The reference number is always backed by a saved row. The system never promises something it has not stored.
The database is the one source of truth; the sheet is a copy	The sheet can be rebuilt at any time. A spreadsheet problem can never lose a lead.
Queue the export inside the database (outbox) instead of a message broker	The database itself holds the retries. No extra server to run, and no message can be lost.
The same key sent twice makes one row	A double click, a retry, or a repeated network request cannot create a duplicate lead.
Clean the data before checking it	Checking raw text can accept text that only looks safe, or reject text that is fine after cleaning. Cleaning first fixes both problems.
Form limits equal the database column sizes	Data that passes the check can never be cut short or rejected by the database.
Check in three places (browser, website server, intake service)	The browser check is for comfort. The other two are the real guard. The server never trusts the browser.
The website server sits in front of the intake service	The intake service does not need to be reachable from the internet. This removes cross-site traffic and shrinks the attack surface.

Decision	Why
Three sizes of rate limit (per minute, burst, all users)	One limit cannot cover both a slow trickle and a sudden flood.
No personal data in the logs	Logs are copied around. Names, e-mails and phones are masked; a message is logged as a length, not its text; an address is stored only as a keyed code.
Hold the database connection only while claiming rows	Slow spreadsheet work cannot use up the database connection pool.
Write the file to a temporary name, then swap	No reader ever sees a half-written file.
No code generators in the intake service	Fewer build tools means fewer build failures. Plain, boring code is safe code.
Keep the pages static and delay the form code	The page paints without waiting for the heavy form code, so the main thread stays free and the page feels instant.

9. Privacy and safety choices

- The real visitor address is never stored. Only a keyed code is stored, so the same address can be recognised without keeping the address.
- Markup and control characters are removed from every field, so stored text can never run as code in the database, in a CSV export, or in a spreadsheet viewer.
- Spreadsheet formulas are defused: a leading = or + is neutralised so a cell cannot run as a formula.
- A hidden field catches simple robots. It gets a fake success and the request is dropped, so the robot stops trying.
- Security headers (content rules, HTTPS rules, frame blocking) are set in one central place.
- In production the service refuses to start with a sample secret, an empty list of allowed sites, or HTTPS turned off.

10. Front-end speed choices

- Pages are built on the server. The product list filters with plain CSS and only a little code.
- The heavy form-checking code is held back until it is really needed, then loaded in a quiet moment. This cut the first-load code by about 20% and moved the checking library out of the first load.
- Before heavy work continues, the code gives the main thread back, so a scroll or a tap is not delayed.
- Every field reserves space for its message, and the form reserves its full height, so the page never jumps.
- The logo and its moving ring use a transform-only animation, which the browser can run off the main thread.

11. How to run and check it

```
docker compose up --build

  website         http://localhost:3000
  intake service  http://localhost:8080
  database        localhost:5432
  excel file      in the docker volume: /app/data/exports/inquiries.xlsx

checks
  backend   :  mvn verify
  frontend  :  npm run typecheck  &&  npm run lint  &&  npm run build
```

12. Known limits and trade-offs

- Rate limiting is kept in memory, so it fits a single intake instance. Running several instances needs a shared counter (Redis or a firewall).
- The Excel file is rewritten for each batch. At a few hundred rows a batch this takes milliseconds; at a very high volume, switch to daily files or the Google Sheet sink.
- The Google Sheets sink needs a webhook address. Without it, only the Excel sink is active.

> In one line: the visitor's inquiry is saved to the database first, the spreadsheet is only a copy, and every part of the path from the browser to the spreadsheet is checked, retried, and safe to run twice.